Workshop – Week 7 – Some selected solutions

1. Create a class to store the data for an item in a shop. The class must contain the following data: name (string), price (float), number in stock (integer)
The class must have a method to load the data and a method to display the data.
Declare an array of 1000 items.

Solution to Q1:

The question did not ask for a main program. However, it is useful to make a small main program just to test the methods. If this question was in a quiz, you would NOT include the main program.

```
#include <iostream>
#include <cstdio>   // needed for getchar
using namespace std;

class item_class {
private:
  string name;
  float price;
  int stock;
public:
  void loaddata();
  void display();
};

item_class items[1000];

int main(){
// a small main just to test the class
  items[0].loaddata();
  items[0].display();
}

//---------- methods for the item_class ----------
void item_class::loaddata() {
  cout << "\nEnter the data for a particular item:\n";
  cout << "Enter the name (e.g. bread) ";
  getline(cin, name);
  cout << "Enter the price (e.g. 2.75) ";
  cin >> price;
  cout << "Enter how many items in stock (e.g. 100) ";
  cin >> stock;
  getchar();  // flush the input buffer
}

void item_class::display() {
  cout << "Item: " << name << "    $" << price;
  cout << "   (" << stock << " in stock)" << endl;
}
```

Note: Question 1 does not actually need the getchar but question 2 is going to need it.

2. Write a C++ program that loads three items into the array set up in question 1, for example: bread, kumara, apple. Input suitable values for the price and number in stock. Display the data for these three items.

Solution to Q2:

```
#include <iostream>
#include <cstdio>   // needed for getchar
using namespace std;

class item_class {
private:
  string name;
  float price;
  int stock;
public:
  void loaddata();
  void display();
};

item_class items[1000];

int main(){
  int i;
  for (i = 0; i < 3; i++) {
    items[i].loaddata();
  }
  cout << "\nFull list of all items follows:\n";
  for (i = 0; i < 3; i++) {
    items[i].display();
  }
}

//---------- methods for the item_class ----------
void item_class::loaddata() {
  cout << "\nEnter the data for a particular item:\n";
  cout << "Enter the name (e.g. bread) ";
  getline(cin, name);
  cout << "Enter the price (e.g. 2.75) ";
  cin >> price;
  cout << "Enter how many items in stock (e.g. 100) ";
  cin >> stock;
  getchar();  // flush the input buffer
}

void item_class::display() {
  cout << "Item: " << name << "    $" << price;
  cout << "   (" << stock << " in stock)" << endl;
}
```

Challenge: remove the getchar and see what happens.

3. Modify the class in (2) above to store the price as an integer value in cents. The price is still entered as a float and displayed as a float but is stored as an integer value. Change the data and the methods in the class as needed. Use your program in (2) to check that your class is portable – you should not have to make any changes to the program in question 2.

This little exercise is rather pointless. However, the main lesson is that classes must be able to be updated without affecting the programs that use them – especially if the class is in a library.

Solution to Q3:

```
#include <iostream>
#include <cstdio>   // needed for getchar
using namespace std;

class item_class {
private:
  string name;
  int price;
  int stock;
public:
  void loaddata();
  void display();
};

item_class items[1000];

int main(){
  int i;
  for (i = 0; i < 3; i++) {
    items[i].loaddata();
  }
  cout << "\nFull list of all items follows:\n";
  for (i = 0; i < 3; i++) {
    items[i].display();
  }
}

//---------- methods for the item_class ----------
void item_class::loaddata() {
  float temp;
  cout << "\nEnter the data for a particular item:\n";
  cout << "Enter the name (e.g. bread) ";
  getline(cin, name);
  cout << "Enter the price (e.g. 2.75) ";
  cin >> temp;
  price = temp * 100;
  cout << "Enter how many items in stock (e.g. 100) ";
  cin >> stock;
  getchar();  // flush the input buffer
}

void item_class::display() {
  float temp;
  temp = (float) price / 100.0;
  cout << "Item: " << name << "    $" << temp;
  cout << "   (" << stock << " in stock)" << endl;
```

```
}
```

4. What is wrong with the following C++ program? Correct the error.

```
#include <iostream>
using namespace std;

class dummy_class {
private:
  int value1;
  float value2;
public:
  void setone(int a) { value1 = a; }
  int getone() { return value1; }
};

dummy_class fish[25];

int main(){
  fish.[3]setone(57);
}
```

Solution to Q4:

The "dot" is in the wrong place. It should be `fish[3].setone(57);`

5. There are 3 errors in the following program. Identify each error and state what needs to be done to correct the error.

```
#include <iostream>
using namespace std;

class fruit_class {
private:
  string pear;
  int banana;
public:
  void setpear(string k) { pear = k; }
  int getbanana() { return banana; }
};

fruit_class *a, *b;

int main(){
  a = new fruit_class;
  a.setpear("hello");
  b->setpear(27);
}
```

Solution to Q5:

Error 1: pointer `b` is being used but does not hold the address of any variable.
Correction: include the line: `b = new fruit_class;`
Warning – this does NOT show an error but could cause random problems to occur

Error 2: `a` is a pointer. It must not use the "dot" symbol in this way: `a.setpear("hello");`
Correction: change this line to be: `a->setpear("hello");`

Error 3: the method `setpear` requires a string as a parameter.
Correction: change the last line to be: `b->setpear("any string");`

6. At the end of the program below, is the address in r the same as the address of m or not? Give reasons for your answer.

```
#include <iostream>
using namespace std;

class tiny_class {
private:
  bool small;
public:
  void setsmall(bool b) { small = b; }
};

tiny_class m;
tiny_class *r;

int main(){
  r = &m;
  r = new tiny_class;
  cout << "Address of m is " << &m << endl;
  cout << "Address stored in r is " << r << endl;
}
```

Solution to Q6:

The address stored in `r` is not the same as the address of `m`. In the first line (`r = &m;`), `r` was set to be the address of `m`. However, on the next line (`r = new tiny_class;`) `r` gets changed to be the address of a new memory location which is set up to be able to store the data required for the tiny_class.